

Tasks: Collect 20–30 research abstracts. Automatically POS-tag them using NLTK. oTreat the tagged data as training data for HMM.

```
import zipfile
import pandas as pd

# 1. Open the arxiv_data.csv (1).zip file and extract its contents
zip_file_path = '/content/arxiv_data.csv (1).zip'
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall('/content/')
print("Extracted 'arxiv_data.csv' from the zip file.")

# 2. Load the extracted CSV file into a pandas DataFrame
df = pd.read_csv('/content/arxiv_data.csv')
print("Loaded 'arxiv_data.csv' into a DataFrame.")

# Check available columns to identify the correct abstract column
print(f"DataFrame columns: {df.columns.tolist()}")

# 3. Select the first 30 abstracts from the 'titles' column (assuming 'titles' contains the
# If there's another column that more accurately represents the abstract, it should be used
selected_abstracts_series = df['titles'].head(30)

# 4. Convert the selected abstracts into a list of strings
abstracts_list = selected_abstracts_series.tolist()

print(f"Selected {len(abstracts_list)} abstracts. Displaying the first 2 abstracts:")
for i, abstract in enumerate(abstracts_list[:2]):
    print(f"\nAbstract {i+1}:\n{abstract[:200]}...") # Displaying first 200 characters for
```

```
Extracted 'arxiv_data.csv' from the zip file.
Loaded 'arxiv_data.csv' into a DataFrame.
DataFrame columns: ['titles', 'summaries', 'terms']
Selected 30 abstracts. Displaying the first 2 abstracts:
```

```
Abstract 1:
Survey on Semantic Stereo Matching / Semantic Depth Estimation...
```

```
Abstract 2:
FUTURE-AI: Guiding Principles and Consensus Recommendations for Trustworthy Artificial Int
```

✓ POS Tagging with NLTK

Subtask:

Tokenize and perform Part-of-Speech (POS) tagging on the collected abstracts using NLTK.

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.tag import pos_tag
```

```
# 1. Download necessary NLTK data
nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('punkt_tab', quiet=True) # Added explicit download for 'punkt_tab' as per p
nltk.download('averaged_perceptron_tagger_eng', quiet=True) # Added explicit download for
print("Downloaded NLTK 'punkt', 'averaged_perceptron_tagger', 'punkt_tab', and 'averaged_

# 2. Initialize an empty list to store the POS-tagged output
tagged_sentences = []

# 3. Iterate through each abstract in the abstracts_list
for abstract in abstracts_list:
    # a. Use nltk.sent_tokenize to split the abstract into sentences
    sentences = sent_tokenize(abstract)
    for sentence in sentences:
        # b. For each sentence, use nltk.word_tokenize to split it into words
        words = word_tokenize(sentence)
        # c. Apply nltk.pos_tag to the list of words to get a list of (word, tag) tuples
        tagged_sentence = pos_tag(words)
        # d. Append the list of (word, tag) tuples to the tagged_sentences list
        tagged_sentences.append(tagged_sentence)

# 4. Print the first few tagged sentences to verify the output
print(f"\nProcessed {len(tagged_sentences)} sentences. Displaying the first 3 tagged sent
for i, tagged_sent in enumerate(tagged_sentences[:3]):
    print(f"\nTagged Sentence {i+1}:")
    print(tagged_sent)
```

Downloaded NLTK 'punkt', 'averaged_perceptron_tagger', 'punkt_tab', and 'averaged_perceptr

Processed 30 sentences. Displaying the first 3 tagged sentences:

Tagged Sentence 1:

[('Survey', 'NNP'), ('on', 'IN'), ('Semantic', 'NNP'), ('Stereo', 'NNP'), ('Matching', 'NN

Tagged Sentence 2:

[('FUTURE-AI', 'NN'), (':', ':'), ('Guiding', 'NNP'), ('Principles', 'NNP'), ('and', 'CC')

Tagged Sentence 3:

[('Enforcing', 'VBG'), ('Mutual', 'JJ'), ('Consistency', 'NNP'), ('of', 'IN'), ('Hard', 'N

```
from nltk.tag.hmm import HiddenMarkovModelTagger
import numpy as np
```

```
# 1. Prepare data for HMM training
# Extract all unique tags and words from the tagged_sentences
all_tags = set()
all_words = set()
for sentence in tagged_sentences:
    for word, tag in sentence:
        all_tags.add(tag)
        all_words.add(word)

tags_list = sorted(list(all_tags))
words_list = sorted(list(all_words))
```

```

# Create mappings from tag/word to index for matrix representation if needed, or use dict
tag_to_idx = {tag: i for i, tag in enumerate(tags_list)}
word_to_idx = {word: i for i, word in enumerate(words_list)}

print(f"Unique tags found: {len(tags_list)}")
print(f"Unique words found: {len(words_list)}")

# 2. Instantiate and train the HiddenMarkovModelTagger
# NLTK's HMMTagger can directly train on tagged sentences. It automatically infers states
# For simplicity, we use a basic HMMTagger. If 'start_prob', 'transitions', 'emissions' a
# the model learns them from the training data.

hmm_tagger = HiddenMarkovModelTagger.train(tagged_sentences)
print("HMM Tagger trained successfully.")

# 3. Access and store transition probabilities
transition_probabilities = {}
for tag1 in tags_list:
    transition_probabilities[tag1] = {}
    for tag2 in tags_list:
        prob = 0.0
        # Access the probability from the _transitions dictionary attribute
        # _transitions[tag1] is a LidstoneProbDist object, use .prob(tag2) to get the prob
        if tag1 in hmm_tagger._transitions:
            prob = hmm_tagger._transitions[tag1].prob(tag2)
        if prob > 0: # Store only non-zero probabilities for compactness
            transition_probabilities[tag1][tag2] = prob

# 4. Access and store emission probabilities
emission_probabilities = {}
for tag in tags_list:
    emission_probabilities[tag] = {}
    for word in words_list:
        prob = 0.0
        # Access the probability from the _outputs dictionary attribute
        # _outputs[tag] is a LidstoneProbDist object, use .prob(word) to get the probabilit
        if tag in hmm_tagger._outputs:
            prob = hmm_tagger._outputs[tag].prob(word)
        if prob > 0: # Store only non-zero probabilities for compactness
            emission_probabilities[tag][word] = prob

print("Transition and Emission probabilities extracted.")

# Display a few examples of transition probabilities
print("\nSample Transition Probabilities:")
count = 0
for tag1, transitions in transition_probabilities.items():
    for tag2, prob in transitions.items():
        if count < 3: # Display only first 3 non-zero probabilities
            print(f"P({tag2}|{tag1}) = {prob:.4f}")
            count += 1
        else:
            break
    if count == 3: break

# Display a few examples of emission probabilities
print("\nSample Emission Probabilities:")

```

```

count = 0
for tag, emissions in emission_probabilities.items():
    for word, prob in emissions.items():
        if count < 3: # Display only first 3 non-zero probabilities
            print(f"P({word}|{tag}) = {prob:.4f}")
            count += 1
        else:
            break
    if count == 3: break

```

Unique tags found: 16
 Unique words found: 183
 HMM Tagger trained successfully.
 Transition and Emission probabilities extracted.

Sample Transition Probabilities:

$P(,|,) = 0.0217$

$P(:|,) = 0.0217$

$P(CC|,) = 0.2391$

Sample Emission Probabilities:

$P(,|,) = 0.1455$

$P(/|,) = 0.0047$

$P(2D|,) = 0.0047$

```
import collections
```

```
# 1. Identify and analyze the most frequent tag transitions
```

```
# Store top transitions for each tag
```

```
top_transitions = {}
```

```
num_top_transitions = 5 # Number of top transitions to display per tag
```

```
for tag1, transitions in transition_probabilities.items():
```

```
    # Sort transitions by probability in descending order
```

```
    sorted_transitions = sorted(transitions.items(), key=lambda item: item[1], reverse=True)
```

```
    top_transitions[tag1] = sorted_transitions[:num_top_transitions]
```

```
print("Top Tag Transitions:")
```

```
print("-----")
```

```
for tag1, transitions in top_transitions.items():
```

```
    print(f"From tag '{tag1}':")
```

```
    for tag2, prob in transitions:
```

```
        print(f"    -> '{tag2}' (Probability: {prob:.4f})")
```

```
    print()
```

Top Tag Transitions:

From tag ',':

-> 'NNP' (Probability: 0.4565)

-> 'CC' (Probability: 0.2391)

-> ',' (Probability: 0.0217)

-> ':' (Probability: 0.0217)

-> 'CD' (Probability: 0.0217)

From tag ':':

-> 'JJ' (Probability: 0.2628)

-> 'NNP' (Probability: 0.2628)

```
-> 'DT' (Probability: 0.1987)
-> 'VBG' (Probability: 0.1346)
-> 'NN' (Probability: 0.0705)
```

From tag 'CC':

```
-> 'NNP' (Probability: 0.8026)
-> ',' (Probability: 0.0132)
-> ':' (Probability: 0.0132)
-> 'CC' (Probability: 0.0132)
-> 'CD' (Probability: 0.0132)
```

From tag 'CD':

```
-> 'NNP' (Probability: 0.6739)
-> ',' (Probability: 0.0217)
-> ':' (Probability: 0.0217)
-> 'CC' (Probability: 0.0217)
-> 'CD' (Probability: 0.0217)
```

From tag 'DT':

```
-> 'JJ' (Probability: 0.4271)
-> 'NNP' (Probability: 0.4271)
-> ',' (Probability: 0.0104)
-> ':' (Probability: 0.0104)
-> 'CC' (Probability: 0.0104)
```

From tag 'IN':

```
-> 'NNP' (Probability: 0.7798)
-> 'JJ' (Probability: 0.1062)
-> 'DT' (Probability: 0.0544)
-> 'VBG' (Probability: 0.0285)
-> ',' (Probability: 0.0026)
```

From tag 'JJ':

```
-> 'NNP' (Probability: 0.4930)
-> 'NN' (Probability: 0.2483)
-> 'CD' (Probability: 0.1084)
-> 'JJ' (Probability: 0.0734)
-> ':' (Probability: 0.0385)
```

From tag 'NN':

```
-> ':' (Probability: 0.5430)
-> 'IN' (Probability: 0.2204)
-> 'NN' (Probability: 0.1129)
-> 'VBG' (Probability: 0.0591)
-> ',' (Probability: 0.0054)
```

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag

# 1. Choose a new, unseen sentence relevant to the domain
test_sentence = "This paper presents a novel approach for image segmentation using deep l

print(f"Test Sentence: {test_sentence}\n")

# 2. Tokenize the test_sentence into words
tokenized_words = word_tokenize(test_sentence)

# 3. Apply the trained hmm_tagger
hmm_tagged_sentence = hmm_tagger.tag(tokenized_words)
```

```
# 4. Apply NLTK's default POS tagger
nltk_default_tagged_sentence = pos_tag(tokenized_words)

# 5. Print both tagged sentences for comparison
print("HMM Tagger Output:")
print(hmm_tagged_sentence)

print("\nNLTK Default Tagger Output:")
print(nltk_default_tagged_sentence)
```

Test Sentence: This paper presents a novel approach for image segmentation using deep learning

HMM Tagger Output:

```
[('This', 'JJ'), ('paper', 'NN'), ('presents', ':'), ('a', 'DT'), ('novel', 'JJ'), ('approach', 'NN'), ('for', 'IN'), ('image', 'NN'), ('segmentation', 'NN'), ('using', 'VBG'), ('deep', 'JJ'), ('learning', 'NN')]
```

NLTK Default Tagger Output:

```
[('This', 'DT'), ('paper', 'NN'), ('presents', 'VBZ'), ('a', 'DT'), ('novel', 'JJ'), ('approach', 'NN'), ('for', 'IN'), ('image', 'NN'), ('segmentation', 'NN'), ('using', 'VBG'), ('deep', 'JJ'), ('learning', 'NN')]
```

```
print("Top Tag Transitions (re-displayed):")
print("-----")
for tag1, transitions in top_transitions.items():
    print(f"From tag '{tag1}':")
    for tag2, prob in transitions:
        print(f"  -> '{tag2}' (Probability: {prob:.4f})")
    print()

# 2. Choose a relevant POS tag (e.g., 'NNP') and print the top 5 emission probabilities
target_tag = 'NNP'
if target_tag in emission_probabilities:
    nnp_emissions = emission_probabilities[target_tag]
    # Sort emissions by probability in descending order
    sorted_nnp_emissions = sorted(nnp_emissions.items(), key=lambda item: item[1], reverse=True)

    print(f"\nTop 5 Emission Probabilities for tag '{target_tag}':")
    print("-----")
    for word, prob in sorted_nnp_emissions[:5]:
        print(f"  P('{word}' | '{target_tag}') = {prob:.4f}")
else:
    print(f"\nNo emission probabilities found for tag '{target_tag}'.")
```

```

-> ',' (Probability: 0.0385)
-> ':' (Probability: 0.0385)
-> 'CC' (Probability: 0.0385)
-> 'CD' (Probability: 0.0385)

```

From tag 'TO':

```

-> 'NNP' (Probability: 0.5833)
-> ',' (Probability: 0.0278)
-> ':' (Probability: 0.0278)
-> 'CC' (Probability: 0.0278)
-> 'CD' (Probability: 0.0278)

```

From tag 'VBD':

```

-> 'IN' (Probability: 0.4231)
-> ',' (Probability: 0.0385)
-> ':' (Probability: 0.0385)
-> 'CC' (Probability: 0.0385)
-> 'CD' (Probability: 0.0385)

```

From tag 'VBG':

```

-> 'NNP' (Probability: 0.4079)
-> 'JJ' (Probability: 0.2763)
-> 'DT' (Probability: 0.1447)
-> ',' (Probability: 0.0132)
-> ':' (Probability: 0.0132)

```

From tag 'VBN':

```

-> 'IN' (Probability: 0.3056)
-> 'NNP' (Probability: 0.3056)
-> ',' (Probability: 0.0278)
-> ':' (Probability: 0.0278)
-> 'CC' (Probability: 0.0278)

```

Top 5 Emission Probabilities for tag 'NNP':

```

-----
P('Segmentation'|'NNP') = 0.1004
P('Image'|'NNP') = 0.0560
P('Medical'|'NNP') = 0.0560
P('Semantic'|'NNP') = 0.0227
P('Transformer'|'NNP') = 0.0227

```

